

Gordon College
Department of Computer Science

Backend Development Laravel + MySQL

Final Project Specifications

A.Y. 2025–2026 | 2nd Semester

This document outlines the official project specifications for the BSCS Backend Development course. Students are expected to implement each project using the Laravel PHP framework with a MySQL database.

Project Overview

The following table summarizes the four projects assigned to different groups in the Backend Development class. Each project must be built using the Laravel PHP framework with a MySQL database, following the MVC pattern and all general requirements outlined in this document.

#	Project Title	Group Size	Stack
1	Student Attendance Tracker	6 members	Laravel + MySQL
2	Library Book Borrowing System	6 members	Laravel + MySQL
3	School Event Management	5 members	Laravel + MySQL
4	Classroom Assignment Submission Portal	5 members	Laravel + MySQL

Project Specifications

□ Project 1: Student Attendance Tracker

□ Project Description

The Student Attendance Tracker is a web-based application designed to assist teachers in efficiently recording, managing, and monitoring student attendance. Rather than relying on manual paper-based logs or fragmented spreadsheets, this system centralizes all attendance data in a structured database, making it easy to retrieve and analyze over time.

Teachers can log daily attendance for each student, mark them as present, absent, or late, and immediately view a running dashboard that summarizes overall class attendance. The system also provides a basic reporting feature that calculates each student's attendance percentage, enabling educators to quickly identify students at risk due to excessive absences.

Built with Laravel and MySQL, the application follows MVC architecture, leverages Eloquent ORM for database operations, and uses Laravel Breeze for secure user authentication. This project is an ideal starting point for students to apply full-stack concepts including routing, form validation, database migrations, and blade templating.

□ Pages & Wireframe Requirements

Login / Register	User authentication page powered by Laravel Breeze. Supports registration for new teacher accounts and secure login for returning users. Includes form validation and CSRF protection.
Dashboard	The main landing page after login. Displays a visual summary of today's attendance, total number of students, overall attendance rate, and recent activity. Acts as the control center for the teacher.
Add Attendance	A form-based page where teachers select a date and mark each student as Present, Absent, or Late. Supports bulk submission for the entire class in a single action.
Student List	A full CRUD interface for managing student records. Teachers can add new students, edit existing information (name, student ID, section), and remove students from the system.
Reports Page	A summary table listing each student alongside their total attendance count and computed attendance percentage. Allows teachers to quickly assess individual and class-wide attendance trends.

□ Key Features

- Full CRUD operations for student records (Create, Read, Update, Delete)
- Daily attendance marking with support for Present, Absent, and Late statuses
- Dashboard showing real-time summary statistics (total students, present today, absent today)
- Attendance percentage calculation per student over a selected time period
- Laravel Breeze authentication (login, register, logout, session management)
- Form validation with user-friendly error messages
- Responsive UI using Blade templates and Bootstrap or Tailwind CSS

□ Scope & Limitations

✔Scope (In)	✗Limitations (Out)
• Attendance recording per class session	• No SMS or email notification system
• CRUD management of student records	• No parent or student-facing portal
• Summary dashboard with attendance statistics	• No grading or academic performance module
• Per-student attendance percentage report	• No integration with external student information systems
• Secure teacher authentication via Laravel Breeze	• No export to PDF or Excel (nice-to-have, not required)

□ Project 2: Library Book Borrowing System

□ Project Description

The Library Book Borrowing System is a web application that digitizes and streamlines the book lending process of a school library. It provides librarians or authorized staff with the tools to manage the library's book inventory, process borrowing and return transactions, and maintain a complete borrowing history for each student.

At its core, the system tracks the availability of each book — whether it is currently on the shelf or checked out by a borrower. When a student borrows a book, the system records the transaction with a timestamp and marks the book as unavailable. When the book is returned, the record is updated accordingly and the book becomes available again. This ensures that staff can always view the real-time status of every title in the library.

The borrower history feature allows staff to look up a student's complete borrowing record, which is useful for accountability and tracking overdue items. The system is built using Laravel's Eloquent relationships (such as one-to-many and many-to-many where appropriate) to model the connection between students, books, and transactions — giving students hands-on experience with relational database design in a real-world context.

□ Pages & Wireframe Requirements

Login / Register	Authentication page for librarians or admin staff. Uses Laravel Breeze for session management and role-based access where applicable.
Dashboard	Provides an overview of the library's current status: total books in inventory, number of books currently borrowed, number of available books, and recent transactions.
Book List	Full CRUD interface for managing the book catalog. Staff can add new book titles (with author, ISBN, and quantity), edit details, and remove titles from the system.
Borrow / Return Form	A transaction form where staff can record a borrow event (selecting a student and a book) or process a return. The form updates book availability in real time.

Borrower History

A paginated log of all borrowing transactions. Filterable by student name or book title. Shows date borrowed, date returned (or "not yet returned"), and current status.

Key Features

- Full CRUD for book records (title, author, ISBN, quantity, availability status)
- Borrow transaction recording with automatic book availability update
- Return processing that marks books as available and timestamps the return date
- Borrower history log — complete transaction records per student
- Dashboard summary showing total books, available books, and currently borrowed books
- Search/filter functionality on the borrower history page
- Laravel Breeze authentication with secure session handling

Scope & Limitations

✓Scope (In)	✗Limitations (Out)
• Book inventory management (CRUD)	• No barcode or QR code scanning
• Borrow and return transaction processing	• No automatic fine or penalty calculation
• Real-time book availability tracking	• No integration with external library catalogs (e.g., OPAC)
• Full borrowing history per student	• No email/SMS reminders for overdue books
• Dashboard with library status overview	• No multi-branch or multi-campus support

Project 3: School Event Management System

Project Description

The School Event Management System is a platform designed to help school administrators or faculty create, organize, and manage school events — while allowing students to browse upcoming events and register their participation. It addresses the common problem of scattered event announcements and manual sign-up sheets by centralizing everything into a single, organized web application.

Administrators can create event entries with full details such as title, description, date, time, venue, and maximum participant capacity. Events can be edited or cancelled as needed. Students, upon logging in, can see a list of all upcoming events and register for those they wish to attend. The system tracks registrations and can display the list of registered participants per event, helping organizers prepare accordingly.

This project challenges students to work with more complex relational data — specifically the many-to-many relationship between students and events (a student can register for multiple events; an event can have multiple

student registrants). Students will use Laravel's pivot tables and Eloquent relationships to model and query this structure, reinforcing key backend development concepts beyond simple one-to-many CRUD.

□ Pages & Wireframe Requirements

Login / Register	Separate authentication flows for Admin (event manager) and Student (event viewer/registrant) roles. Role-based middleware ensures that only admins can manage events.
Dashboard	Displays upcoming events in a card or list format with key details. Shows how many events are scheduled this week/month and quick links to manage events or view registrations.
Event List (Admin)	Full CRUD interface for events. Admins can add a new event (title, description, date, venue, capacity), edit or update event details, and delete past or cancelled events.
Event Details	A dedicated page for each event showing complete information: title, description, date, time, venue, registered participants list, and available slots remaining.
Registration Page	Allows logged-in students to register for an event with a single click. Shows confirmation upon successful registration. Prevents duplicate registration for the same event.

□ Key Features

- Role-based access control: Admin (manage events) vs. Student (view and register)
- Full CRUD for events (title, description, date/time, venue, capacity)
- Student event registration with duplicate prevention
- Participant list per event visible to admins
- Dashboard showing upcoming events with a count of registrants
- Event capacity enforcement — prevent registration if event is full
- Laravel Breeze authentication with middleware-protected routes

□ Scope & Limitations

✓Scope (In)	✗Limitations (Out)
• Event creation, editing, and deletion (Admin)	• No ticketing, QR codes, or payment processing
• Student event registration and deregistration	• No calendar sync (Google Calendar, iCal)
• Participant tracking per event	• No push notifications or email reminders
• Role-based access (Admin vs. Student)	• No attendance marking on the day of the event
• Dashboard with upcoming event overview	• No publicly accessible (unauthenticated) event listings

□ Project 4: Classroom Assignment Submission Portal

□ Project Description

The Classroom Assignment Submission Portal is a web-based system that facilitates the digital submission and management of academic assignments between teachers and students. It eliminates the need for physical handoffs or unstructured file sharing through messaging apps by providing a structured, organized environment for posting assignments and tracking submissions.

Teachers can create and manage assignments — specifying the subject, instructions, and deadline — and view all submissions received from students. Students, upon logging in, can browse their active assignments, upload their output files (documents, images, or compressed archives), and track whether their submission has been received. The system stores all submitted files and maintains a submission history that teachers can review at any time.

This project introduces students to file handling in Laravel — a critical real-world skill. They will use Laravel's Storage facade to manage file uploads securely, handle file validation (type and size restrictions), and organize uploaded files in a structured directory. The submission history feature reinforces the use of Eloquent relationships to link assignments, students, and their respective submissions in a normalized database schema.

□ Pages & Wireframe Requirements

Login / Register	Role-based authentication for Teachers and Students using Laravel Breeze. Middleware ensures teachers access management views and students access submission views.
Dashboard	For teachers: a list of all posted assignments with submission counts. For students: a list of active assignments they need to submit, highlighting upcoming deadlines.
Assignment List (Teacher)	Full CRUD interface for managing assignments. Teachers can create a new assignment (title, description, subject, due date), edit details, and delete old assignments.
Assignment Upload Form (Student)	A file submission form where students select an assignment and upload their output file. Includes file type/size validation. Confirms successful upload with a success message.
Submission History	A paginated table showing all submissions per assignment or per student. Displays student name, assignment title, file name, and submission timestamp. Accessible to teachers only.

□ Key Features

- Role-based access: Teacher (post and manage assignments) vs. Student (view and submit)
- Full CRUD for assignments (title, description, subject, due date)
- File upload functionality using Laravel's Storage facade

- File validation (restrict to accepted file types and maximum file size)
- Submission history per assignment — visible to teachers with timestamps
- Prevent duplicate submissions: students can replace/re-upload but not submit twice accidentally
- Deadline awareness — visually indicate past-due assignments on the student dashboard

□ Scope & Limitations

✓Scope (In)	✗Limitations (Out)
• Assignment posting and management (Teacher CRUD)	• No grading, feedback, or annotation system
• File upload submission by students	• No plagiarism detection or similarity checking
• Submission tracking with timestamps	• No cloud storage integration (S3, Google Drive)
• Role-based views (Teacher vs. Student)	• No email notifications for new assignments or submissions
• File type and size validation	• No large file support beyond server upload limits

□ General Project Requirements

All projects, regardless of topic, must comply with the following technical and academic standards:

- Built with Laravel (latest stable version) and MySQL as the primary database
- Must follow MVC (Model-View-Controller) architecture strictly
- Use Eloquent ORM for all database interactions — raw SQL queries should be avoided
- Authentication implemented using Laravel Breeze
- All forms must include server-side validation using Laravel's validation rules
- Database must be designed using proper normalization (at minimum, 1NF to 3NF)
- Migrations and seeders must be included for database setup
- Codebase must be version-controlled using Git (GitHub repository submission required)
- README file must include setup instructions, feature list, and group member names
- Live demo must be presented to the instructor during the final submission week

□ Grading Rubric (General)

Criteria	Weight	Description
Functionality — Core features work as specified	40%	All required features are functional, forms validate, CRUD works, auth is implemented.

Code Quality & Laravel Best Practices	20%	Proper MVC structure, use of Eloquent, clean migrations, no raw SQL, meaningful naming.
Database Design	15%	Normalized schema, correct relationships, proper use of migrations and seeders.
UI/UX & Blade Templates	10%	Clean and usable interface, Blade layouts used consistently, responsive design.
Documentation & README	10%	Setup instructions, feature list, ERD or schema diagram, and group member names.
Live Demo & Presentation	5%	Group can run and demonstrate the app live and answer basic questions about their code.